PYTHON FOR BEGINNERS · SESSION 16

NumPy Basics

Arrays, shape, dtype, indexing and slicing.

Recap

Ask the class:

- What is the difference between `try`, `except`, `else` and `finally`?
- What error does `int("abc")` cause?
- How do you read a traceback: which line first?

What is NumPy?

NumPy (Numerical Python) is a package for working with **numbers in bulk**. Its main tool is the **array**: a grid of numbers, all of the same type.

- **Fast:** it does the loops in fast compiled code, not in slow Python.
- **Short:** one line works on a million numbers at once.
- **The base** of `pandas`, `matplotlib`, `scikit-learn` and most data science and machine learning tools.

Install it once on your computer (terminal)

```
pip install numpy
```

By convention we import it with the short name `np`.

```
import numpy as np

print(np.__name__)
print(type(np.array([1, 2, 3])))
```

OUTPUT

```
numpy
<class 'numpy.ndarray'>
```

NumPy in the Try it boxes

The first time a box imports NumPy, the page downloads it (a few seconds). After that it is quick. The browser version of Python is a little different from the one on your computer, so one or two outputs below (marked in the notes) may differ slightly.

Lists vs arrays

A list repeats when you multiply it. An array does **maths on every item**.

```
import numpy as np

nums = [1, 2, 3]
arr = np.array([1, 2, 3])

print(nums * 2)
print(arr * 2)
print(nums + nums)
print(arr + arr)
```

OUTPUT

```
[1, 2, 3, 1, 2, 3]
[2 4 6]
[1, 2, 3, 1, 2, 3]
[2 4 6]
```

This is called **vectorised** (or element-wise) maths. No loop needed.

```
import numpy as np

prices = np.array([100, 250, 80, 400])

print(prices * 1.18)
print(prices - 20)
print(prices > 200)
```

OUTPUT

```
[118.  295.   94.4 472. ]
[ 80 230  60 380]
[False  True False  True]
```

	LIST	NUMPY ARRAY
items	any mix of types	all the **same** type
`* 2`	repeats the list	doubles each number
speed	slower for big data	much faster
maths on all items	needs a loop	one line

Creating arrays

From a list

```
import numpy as np

a = np.array([10, 20, 30, 40])
print(a)

b = np.array([[1, 2, 3],
              [4, 5, 6]])
print(b)
```

OUTPUT

```
[10 20 30 40]
[[1 2 3]
 [4 5 6]]
```

- A list of numbers makes a **1D** array (a row).
- A list of lists makes a **2D** array (a table with rows and columns).
- Printing an array shows the numbers **without commas**.

Ready-made arrays

FUNCTION	WHAT IT MAKES
`np.zeros(5)`	five `0.0`s
`np.ones((2, 3))`	2 rows and 3 columns of `1.0`
`np.full((2, 2), 7)`	a 2 by 2 grid of `7`
`np.arange(0, 10, 2)`	like `range`, as an array
`np.linspace(0, 1, 5)`	5 evenly spaced numbers from 0 to 1
`np.eye(3)`	the 3 by 3 identity grid

```
import numpy as np

print(np.zeros(5))
print(np.ones((2, 3)))
print(np.full((2, 2), 7))
print(np.arange(0, 10, 2))
print(np.linspace(0, 1, 5))
print(np.eye(3))
```

OUTPUT

```
[0. 0. 0. 0. 0.]
[[1. 1. 1.]
 [1. 1. 1.]]
[[7 7]
 [7 7]]
[0 2 4 6 8]
[0.   0.25 0.5  0.75 1.  ]
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

arange vs linspace

`arange(start, stop, step)` is told the **step** and the stop is **not** included. `linspace(start, stop, count)` is told **how many numbers** and the stop **is** included.

Array attributes

Every array knows facts about itself. They are **attributes**, so there are **no brackets**.

ATTRIBUTE	MEANING
`a.ndim`	number of dimensions (1, 2, 3 ...)
`a.shape`	size in each dimension, as a tuple
`a.size`	total number of items
`a.dtype`	the data type of the items
`a.itemsize`	bytes used by one item

```
import numpy as np

a = np.array([[1, 2, 3],
              [4, 5, 6]])

print(a.ndim)
print(a.shape)
print(a.size)
print(len(a))
```

OUTPUT

```
2
(2, 3)
6
2
```

- `shape` is `(2, 3)`: **2 rows** and **3 columns**.
- `size` is `2 * 3 = 6`.
- `len(a)` gives only the length of the **first** dimension (the number of rows).

Understanding shape

ARRAY	SHAPE	PICTURE
`np.array([1, 2, 3])`	`(3,)`	a row of 3
`np.array([[1, 2, 3]])`	`(1, 3)`	1 row, 3 columns
`np.array([[1], [2], [3]])`	`(3, 1)`	3 rows, 1 column
`np.zeros((2, 3, 4))`	`(2, 3, 4)`	2 tables of 3 rows and 4 columns

```
import numpy as np

print(np.array([1, 2, 3]).shape)
print(np.array([[1, 2, 3]]).shape)
print(np.array([[1], [2], [3]]).shape)
print(np.zeros((2, 3, 4)).shape)
```

OUTPUT

```
(3,)
(1, 3)
(3, 1)
(2, 3, 4)
```

The comma in (3,)

A tuple with one item is written `(3,)`. The comma tells Python it is a tuple. So `(3,)` means a **1D array with 3 items**.

Work out the shape first, then press Run to check

```
import numpy as np

a = np.zeros((4, 5))
print(a.shape)
print(a.size)
print(a.ndim)
```

dtype: the data type

All items in an array share **one type**, called the `dtype`. This is what makes arrays fast and compact.

DTYPE	MEANING
`int8`, `int16`, `int32`, `int64`	whole numbers, using 1, 2, 4 or 8 bytes
`float32`, `float64`	decimals (`float64` is the usual one)
`bool`	`True` or `False`
`<U5`	text, up to 5 characters

```
import numpy as np

print(np.array([1.5, 2.5, 3.5]).dtype)
print(np.array([True, False]).dtype)
print(np.array(["cat", "horse"]).dtype)
print(np.array([1, 2, 3], dtype=np.int8).dtype)
print(np.array([1, 2, 3], dtype=np.float64).dtype)
```

OUTPUT

```
float64
bool
<U5
int8
float64
```

Whole numbers: int64 or int32?

On most computers `np.array([1, 2, 3]).dtype` is `int64`. In the browser version of Python used by these boxes it can show `int32`. Both are whole numbers. To be sure, ask for the type you want with `dtype=`.

Mixed types become one type

```
import numpy as np

print(np.array([1, 2.5, 3]))
print(np.array([1, 2.5, 3]).dtype)
print(np.array([1, "a", 3]))
print(np.array([True, 2, 3.5]))
```

OUTPUT

```
[1.  2.5 3. ]
float64
['1' 'a' '3']
[1.  2.  3.5]
```

- NumPy picks a type that fits **everything**: ints and floats become floats.
- Text and numbers together become **text**.
- A `True` becomes `1.0` when mixed with floats.

Changing the type: astype

```
import numpy as np

a = np.array([1.9, 2.5, 3.1])
print(a.astype(int))
print(a.astype(str))

b = np.array([0, 1, 2, 0])
print(b.astype(bool))
```

OUTPUT

```
[1 2 3]
['1.9' '2.5' '3.1']
[False  True  True False]
```

`astype` makes a **new** array. Turning floats into ints **cuts off** the decimals (it does not round).

reshape, flatten and transpose

`reshape` gives the **same numbers** a new shape. The new shape must hold the same **total count**.

```
import numpy as np

a = np.arange(1, 13)
print(a)

print(a.reshape(3, 4))
print(a.reshape(4, 3))
print(a.reshape(2, 2, 3).shape)
```

OUTPUT

```
[ 1  2  3  4  5  6  7  8  9 10 11 12]
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]
(2, 2, 3)
```

- 12 numbers can be `3 x 4`, `4 x 3`, `2 x 6` or `2 x 2 x 3`, but **not** `5 x 3`.
- Use `-1` for "work this size out for me": `a.reshape(3, -1)`.

```
import numpy as np

a = np.arange(1, 7).reshape(2, 3)
print(a)
print(a.T)
print(a.T.shape)
print(a.flatten())
print(a.reshape(-1))
```

OUTPUT

```
[[1 2 3]
 [4 5 6]]
[[1 4]
 [2 5]
 [3 6]]
(3, 2)
[1 2 3 4 5 6]
[1 2 3 4 5 6]
```

NAME	WHAT IT DOES
`a.reshape(r, c)`	new shape, same numbers
`a.T`	**transpose**: rows become columns
`a.flatten()`	turn into a 1D array (a copy)
`a.reshape(-1)`	also 1D, with the size worked out

Indexing a 1D array

Indexing a 1D array works like a list: position **starts at 0**, and **negative** positions count from the end.

```
import numpy as np

a = np.array([10, 20, 30, 40, 50])

print(a[0])
print(a[2])
print(a[-1])
print(a[-2])
```

OUTPUT

```
10
30
50
40
```

You can **change** an item by assigning to it.

```
import numpy as np

a = np.array([10, 20, 30])
a[1] = 99
print(a)

a[0] = 5.7
print(a)
```

OUTPUT

```
[10 99 30]
[ 5 99 30]
```

The dtype stays the same

The array is a whole-number array, so `5.7` was **cut** to `5`. Choose the right `dtype` at the start if you need decimals.

Indexing a 2D array

In a 2D array you give the **row**, then the **column**: `a[row, column]`.

```
import numpy as np

a = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

print(a[0, 0])
print(a[1, 2])
print(a[2, 1])
print(a[-1, -1])
```

OUTPUT

```
1
6
8
9
```

- `a[1, 2]` means row `1`, column `2`, which is `6`.
- `a[1][2]` also works (index twice), but `a[1, 2]` is the NumPy way and is faster.

One whole row

```
import numpy as np

a = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

print(a[0])
print(a[2])
print(a[1][1])
```

OUTPUT

```
[1 2 3]
[7 8 9]
5
```

Work out the output first, then press Run to check

```
import numpy as np

a = np.array([[10, 20, 30],
              [40, 50, 60]])

print(a[1, 0])
print(a[0, -1])
print(a[1])
```

Slicing a 1D array

A slice takes a **part** of an array: `a[start:stop:step]`. The `stop` position is **not included**.

```
import numpy as np

a = np.arange(10, 100, 10)
print(a)

print(a[2:5])
print(a[:3])
print(a[6:])
print(a[::2])
print(a[::-1])
print(a[-3:])
```

OUTPUT

```
[10 20 30 40 50 60 70 80 90]
[30 40 50]
[10 20 30]
[70 80 90]
[10 30 50 70 90]
[90 80 70 60 50 40 30 20 10]
[70 80 90]
```

SLICE	MEANING
`a[2:5]`	positions 2, 3, 4
`a[:3]`	the first 3
`a[6:]`	from position 6 to the end
`a[::2]`	every second item
`a[::-1]`	reversed
`a[-3:]`	the last 3

A slice is a view, not a copy

This is the **big difference from lists**. A NumPy slice **shares memory** with the original. Change the slice and the original changes too.

Work out the output first, then press Run to check

```
import numpy as np

a = np.array([1, 2, 3, 4, 5])
part = a[1:4]
part[0] = 99

print(part)
print(a)
```

Use `.copy()` when you want an independent array.

```
import numpy as np

a = np.array([1, 2, 3, 4, 5])
safe = a[1:4].copy()
safe[0] = 99

print(safe)
print(a)
```

OUTPUT

```
[99  3  4]
[1 2 3 4 5]
```

Slicing a 2D array

Give a slice for the **rows**, a comma, then a slice for the **columns**: `a[rows, columns]`.

```
import numpy as np

a = np.arange(1, 17).reshape(4, 4)
print(a)

print(a[0:2, 1:3])
print(a[:, 0])
print(a[1, :])
print(a[2:, 2:])
```

OUTPUT

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]
 [13 14 15 16]]
[[2 3]
 [6 7]]
[ 1  5  9 13]
[5 6 7 8]
[[11 12]
 [15 16]]
```

SLICE	MEANING
`a[0:2, 1:3]`	rows 0 and 1, columns 1 and 2
`a[:, 0]`	every row, column 0: **the first column**
`a[1, :]`	row 1, every column: **the second row**
`a[2:, 2:]`	from row 2 down and column 2 across

The colon means "all"

A lone `:` means **everything in that direction**. `a[:, 1]` reads as "all rows, column 1". Say it aloud as you write it.

Changing a block

```
import numpy as np

a = np.zeros((3, 4), dtype=np.int8)
a[1, :] = 5
a[:, 3] = 9
a[0:2, 0:2] = 1
print(a)
```

OUTPUT

```
[[1 1 0 9]
 [1 1 5 9]
 [0 0 0 9]]
```

Boolean and fancy indexing

A comparison on an array gives an array of `True` and `False`. Use it as a **filter**.

```
import numpy as np

marks = np.array([45, 82, 67, 91, 38, 74])

passed = marks >= 50
print(passed)
print(marks[passed])
print(marks[marks > 80])
print(marks[(marks > 40) & (marks < 80)])
```

OUTPUT

```
[False  True  True  True False  True]
[82 67 91 74]
[82 91]
[45 67 74]
```

Use & and |, with brackets

Combine conditions with `&` (and) and `|` (or), and put **each condition in brackets**. The words `and` and `or` do not work on whole arrays.

Change items that match

```
import numpy as np

marks = np.array([45, 82, 67, 91, 38, 74])
marks[marks < 50] = 50
print(marks)

print((marks > 70).sum())
```

```
OUTPUT
[50 82 67 91 50 74]
3
```

`True` counts as `1`, so `(marks > 70).sum()` counts **how many** items match.

Fancy indexing: a list of positions

```
import numpy as np

a = np.array([10, 20, 30, 40, 50])
print(a[[0, 2, 4]])
print(a[[4, 0]])

b = np.arange(12).reshape(3, 4)
print(b[[0, 2]])
```

```
OUTPUT
[10 30 50]
[50 10]
[[ 0  1  2  3]
 [ 8  9 10 11]]
```

Unlike a slice, fancy and boolean indexing give a **copy**, not a view.

Maths on arrays

NAME	WHAT IT GIVES
`a.sum()`	total
`a.mean()`	average
`a.max()` / `a.min()`	biggest / smallest
`a.argmax()` / `a.argmin()`	**position** of the biggest / smallest
`np.sort(a)`	a sorted copy
`a.std()`	standard deviation

```
import numpy as np

marks = np.array([45, 82, 67, 91, 38, 74])

print(marks.sum())
print(marks.mean())
print(marks.max(), marks.min())
print(marks.argmax())
print(np.sort(marks))
```

OUTPUT

```
397
66.16666666666667
91 38
3
[38 45 67 74 82 91]
```

axis: rows or columns

On a 2D array, `axis=0` works **down the columns** and `axis=1` works **across the rows**.

```
import numpy as np

a = np.array([[1, 2, 3],
              [4, 5, 6]])

print(a.sum())
print(a.sum(axis=0))
print(a.sum(axis=1))
print(a.mean(axis=0))
```

OUTPUT

```
21
[5 7 9]
[ 6 15]
[2.5 3.5 4.5]
```

- `axis=0` collapses the **rows**, so you get one answer per **column**: `[5 7 9]`.
- `axis=1` collapses the **columns**, so you get one answer per **row**: `[ 6 15]`.

Two arrays together

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([10, 20, 30])

print(a + b)
print(a * b)
print(b / a)
print(a ** 2)
print(np.sqrt(np.array([4, 9, 16])))
```

OUTPUT

```
[11 22 33]
[10 40 90]
[10. 10. 10.]
[1 4 9]
[2. 3. 4.]
```

Arrays of the **same shape** are combined **item by item**. An array and a single number also work: the number is used for every item. This is called **broadcasting**.

Example: a marks table

Rows are students, columns are subjects (maths, science, english).

```
import numpy as np

marks = np.array([
    [85, 90, 78],
    [70, 65, 80],
    [92, 88, 95],
    [55, 60, 48],
])

print("Shape:", marks.shape)
print("Student totals:", marks.sum(axis=1))
print("Subject averages:", marks.mean(axis=0))
print("Best maths mark:", marks[:, 0].max())
print("Top student (row):", marks.sum(axis=1).argmax())
print("Marks below 50:", marks[marks < 50])
```

OUTPUT

```
Shape: (4, 3)
Student totals: [253 215 275 163]
Subject averages: [75.5  75.75 75.25]
Best maths mark: 92
Top student (row): 2
Marks below 50: [48]
```

- `marks[:, 0]` is the **maths column**.
- `sum(axis=1)` adds each **student**. `mean(axis=0)` averages each **subject**.
- `argmax()` gives the **position** of the best total, which is the student number (starting at `0`).

Your turn: add 5 bonus marks to every mark, then print the new student totals

```
import numpy as np

marks = np.array([
    [85, 90, 78],
    [70, 65, 80],
])
```

Common mistakes

- **Forgetting** `import numpy as np`**.** You get `NameError: name 'np' is not defined`.
- **Expecting a slice to be a copy.** Changing it changes the original. Use `.copy()`.
- **Writing** `a(0)` **instead of** `a[0]`**.** Indexing uses **square** brackets.
- **Writing** `a.shape()`**.** `shape` is an attribute, with no brackets.
- **Using** `and` **/** `or` **on arrays.** Use `&` and `|` with brackets.
- **A wrong** `reshape`**.** The total number of items must stay the same.

- **Mixing up `axis=0` and `axis=1`.** Remember: `axis=0` goes **down**, `axis=1` goes **across**.
- **Putting different shapes together.** `(3,)` and `(4,)` cannot be added.

Try it: this fails on purpose

```
import numpy as np

a = np.arange(12)
print(a.reshape(5, 3))
```

```
ERROR
  File "your code", line 4, in <module>
ValueError: cannot reshape array of size 12 into shape (5,3)
```

Try it: this also fails on purpose

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([1, 2, 3, 4])
print(a + b)
```

```
ERROR
  File "your code", line 5, in <module>
ValueError: operands could not be broadcast together with shapes (3,) (4,)
```

Practice

Your turn: make the numbers 1 to 20 as a 4 by 5 array and print its shape

```
import numpy as np


```

Your turn: print the last column and the first row

```
import numpy as np

a = np.arange(1, 13).reshape(3, 4)
```

Your turn: print only the even numbers of the array

```
import numpy as np

a = np.arange(1, 11)
```

Homework and wrap-up

- Make an array of the numbers `0` to `99`, reshape it to `10 x 10`, and print the **diagonal** items (`a[0, 0]`, `a[1, 1]` ...) using a loop.
- Create a `5 x 5` array of zeros, then set the **border** items to `1` using slices.
- Store the marks of 5 students in 4 subjects. Print each student's **average**, each subject's **highest** mark, and **who has the best total**.
- Given daily temperatures in an array, print how many days were **above the average**.
- Show that a slice is a **view** by changing it, then repeat with `.copy()`.
- Revise: `np.array`, `shape`, `ndim`, `size`, `dtype`, `reshape`, indexing, slicing, boolean masks, `axis`.

Next class

More NumPy, or Pandas basics: tables of data with Series and DataFrame.